

Assessment Submission

The code is all in `solution.py`. It has no dependencies outside the standard library, and running `python solution.py` just prints the results of a few quick checks I left at the bottom.

Problem 1: LRU Cache

The thing that makes an LRU cache tricky is that you actually need two different abilities at the same time. You need to find a value by its key really fast, and you also need to know, at any moment, which key has gone the longest without being touched, so you can throw it out when you run out of room.

A plain dictionary solves the first half. Lookup by key is constant time, but a dict doesn't remember the order things were used in, so figuring out the least-recently-used key would mean scanning everything. A linked list solves the second half: if I keep the most-recently-used item at the front and the oldest at the back, then the thing to evict is always sitting right at the tail. The catch is that searching a linked list for a particular key is slow.

So I use both together and let each one cover the other's weakness. The dictionary maps every key to its actual node object inside the list. When a `get` comes in, I look the node up in the dict (fast), then because I'm already holding the node, I can pop it out of wherever it is and stitch it back in at the front without walking the list at all. Removing a node from the middle is cheap precisely because the list is *doubly* linked: each node knows the one before and after it, so I can rewire its two neighbours directly.

One detail I think is worth calling out is the two dummy nodes I keep at the head and tail. They never hold real data. They exist so that every genuine node always has something on both sides of it, which means I never have to write "if this is the first node" or "if the list is empty" branches. All the pointer juggling becomes the same four lines every time, and that's where most of the bugs in this kind of code usually hide.

A few cases I made sure work: with capacity 1, putting a second key kicks the first one out as you'd expect. Putting a key that already exists just overwrites its value and bumps it to the front without growing the cache. A `get` for a key that isn't there returns -1 and leaves everything alone. And I evict *before* I insert the new entry, so the cache never even briefly holds more than its limit.

Problem 2: Event Scheduler

`can_attend_all` is the easy one. If I sort the events by start time, then one person can do all of them as long as each event starts no earlier than the previous one finished. The only wrinkle is the rule that an event ending at 10 and another starting at 10 are fine, so I compare with a strict less-than: a clash only happens when something starts *before* the previous end, not exactly on it.

`min_rooms_required` took me a second to convince myself of, but it comes down to one observation: the number of rooms you need is just the largest number of events that are ever going on at the same time. Think about the single busiest moment in the day. Every event happening right then has to be in its own room, so you can't possibly use fewer rooms than that. And once you've got that many rooms, you're fine the rest of the time, because things are quieter everywhere else. So the answer is really "what's the worst pile-up?"

To find that pile-up I sort by start time and walk through the events while keeping a min-heap of the end times of the rooms currently in use. For each new event I peek at the heap: if the room that frees up soonest is already done by the time this event starts, I reuse it (pop the old end time off). Otherwise the event genuinely overlaps everything that's running, so it needs a fresh room. Either way I push this event's end time on. The size of the heap is how many rooms are busy at that point, and the biggest the heap ever gets is the answer. The reuse-on-equal-time case is again handled by checking `<=`, which keeps back-to-back meetings in the same room.

I checked it against an empty list (zero rooms, and `can_attend_all` is vacuously true), back-to-back meetings like (9,10) and (10,11) which should share one room, a nested pair like (9,12) and (10,11) which needs two, and a single event which needs one.

Final Discussion

1. Complexity of each function.

For the cache, everything is constant time. `get` and `put` each do one dict lookup and a fixed amount of pointer rewiring, so $O(1)$ time and $O(1)$ extra space per call. The helpers `_remove` and `_push_front` are obviously $O(1)$. The cache as a whole holds at most `capacity` entries, so its space is $O(\text{capacity})$.

For the scheduler, both functions are dominated by the sort, so $O(n \log n)$ time. `can_attend_all` then does a single linear pass. `min_rooms_required` does n heap pushes and pops, each $O(\log n)$, which is the same $O(n \log n)$ order. Both use $O(n)$ extra space — the sorted copy of the events, plus the heap in the second one.

2. Why a hash map and a doubly linked list go together.

I touched on this above but to state it plainly: the dict is what makes lookup fast, and the list is what makes the ordering cheap to maintain. If I only had a dict, I'd have no idea which key was the oldest without checking all of them, so eviction would be $O(n)$. If I only had a list, I'd have the ordering for free but I couldn't find a given key without scanning, so `get` would be $O(n)$. Putting them together, the dict turns a key into a direct handle on a list node, and the list turns that node into an $O(1)$ move-to-front or remove. They each erase the other's slow path. And it has to be a *doubly* linked list specifically, because to unlink a node in constant time you need to reach the node before it, which means every node has to carry a pointer backwards as well as forwards.

3. Handing out named rooms.

Right now I only count rooms; I don't say which event goes where. To actually label them I'd keep two things instead of one heap. First, a pool of free room names — a plain stack works, or a min-heap if I want the labels handed out in a tidy order like A, B, C.

Second, a heap of the rooms currently occupied, but now each entry is a pair of (end time, room name) rather than just an end time.

The loop is the same shape as before. For each event I first release any rooms whose end time has passed, pushing their names back into the free pool. Then I grab a name off the free pool for the new event — or, if the pool happens to be empty, I invent a brand new name like “Room D”. I record which name this event got in a result dictionary, and I push (this event’s end, that name) onto the occupied heap. Names get recycled the instant a meeting ends, so the total number of distinct room names I ever create comes out to exactly the peak-overlap number from part 2. The running time doesn’t change; it’s still $O(n \log n)$.

4. Making the cache thread-safe.

The part that’s easy to get wrong here is assuming `get` is safe to run concurrently because it’s “just a read.” It isn’t — `get` reshuffles the list to mark the key as recently used, so two `gets` racing each other can corrupt the pointers exactly like two `puts` would. So reads need locking too.

The straightforward fix is a single lock around the body of both `get` and `put` (I’d reach for `threading.Lock`, or `RLock` if I ever had one method call another). Each operation touches several pointers plus the dict, and all of that has to happen as one indivisible unit, so the lock has to wrap the whole thing, not just a line or two. This is correct and dead simple to reason about. The downside is that it lets only one thread use the cache at a time, so under heavy traffic the lock itself becomes the bottleneck and the threads mostly sit around waiting.

If that contention actually showed up in profiling, the usual next step is to split the cache into several independent shards, each with its own lock, dict, and list, and route a key to a shard by its hash. Then operations on keys in different shards don’t block each other and throughput goes up roughly in proportion to the number of shards. The price you pay is that the LRU ordering is now only correct within each shard rather than globally, which is normally an acceptable approximation, and the code gets more involved. Trying to lock individual nodes instead doesn’t really help, because almost every operation also touches the shared head and tail and the dict, so the contention just moves onto those instead. I’d start with the single lock for correctness and only move to sharding if the numbers said I had to.